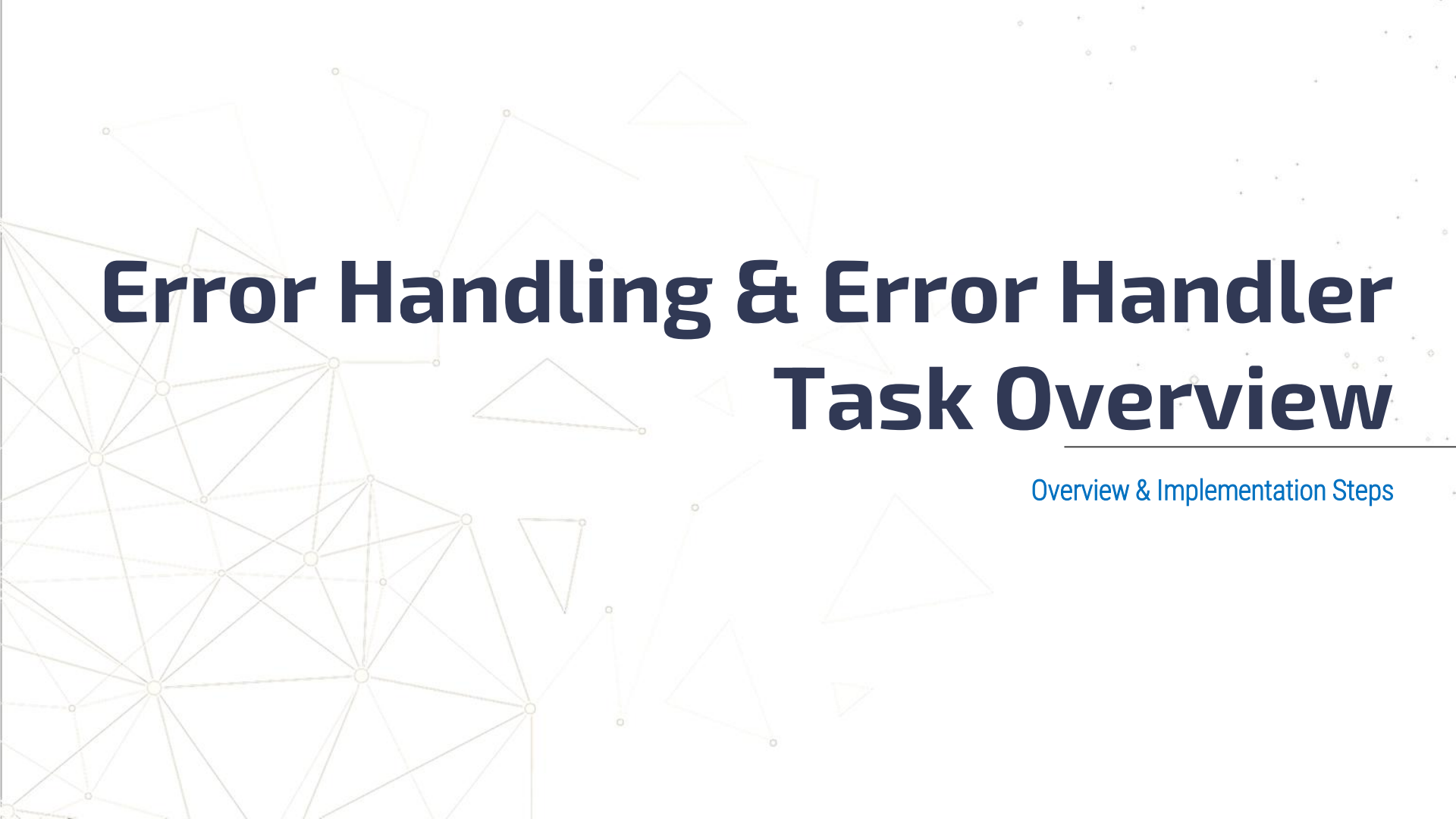




Error Handling & Error Handler Task Overview

Overview & Implementation Steps

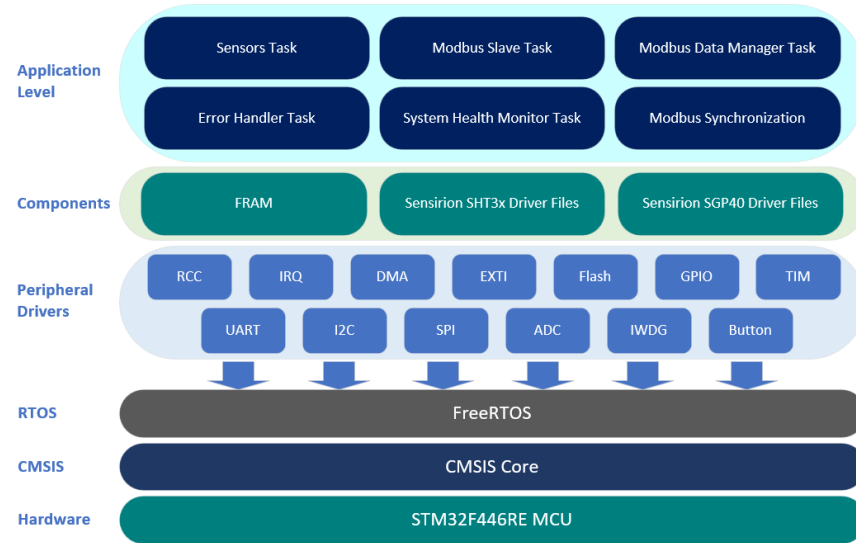
Error Handling Overview

- Error Handling & Error Handler Task Overview
- Error Type Definitions Header File
- Error Handler Header File
- Error Handler Source File
- Error Handler Testing

Error Handling & Error Handler Task Overview Part I

General

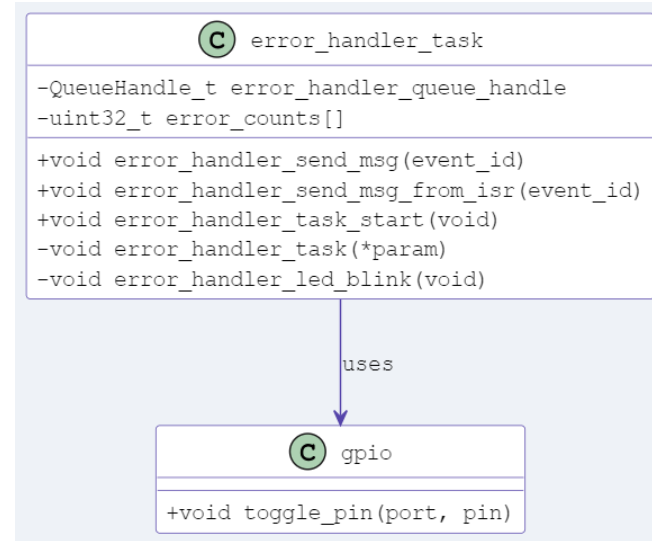
- **Purpose & Functionality:** The Error Handler Task serves as a centralized monitoring and response system for managing errors across various application tasks. The task dynamically detects, logs, and handles errors, providing robust error management.
- **Project Significance:**
 - A dedicated Error Handler Task simplifies debugging during development but also enhances system stability and safety in production. Enables scalable and manageable code.



Error Handling & Error Handler Task Overview Part II

Error Handler Task: Waits for error messages on a queue, processes each received error, and features the following.

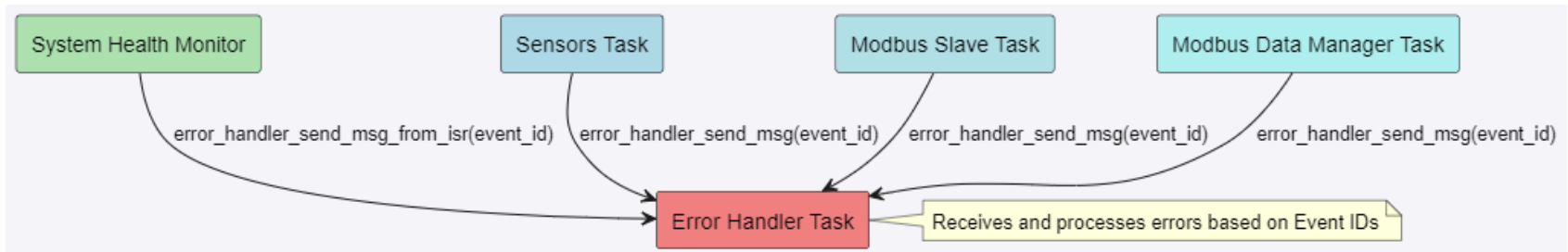
- **Message Handling:** Utilizes [xQueueReceive](#) to await errors, ensuring errors are processed as they occur.
- **Message Sending:** [error_handler_send_msg](#) for typical operations and [error_handler_send_msg_from_isr](#) for interrupt-driven messages.
- **Debugging Aids:** [error_handler_led_blink](#) blinks the USER LED for visual feedback on error detection, and an error counter array for tracking errors for each type.
- **Project Significance:**
 - The task's design allows for easy modifications and scalability, accommodating different types of errors and responses as required by the application.



Error Handling & Error Handler Task Overview Part III

Error Handler Task Communication: Facilitated through (`error_handler_send_msg` and `error_handler_send_msg_from_isr`), allowing other tasks to send messages to the Error Handler Task when errors are detected.

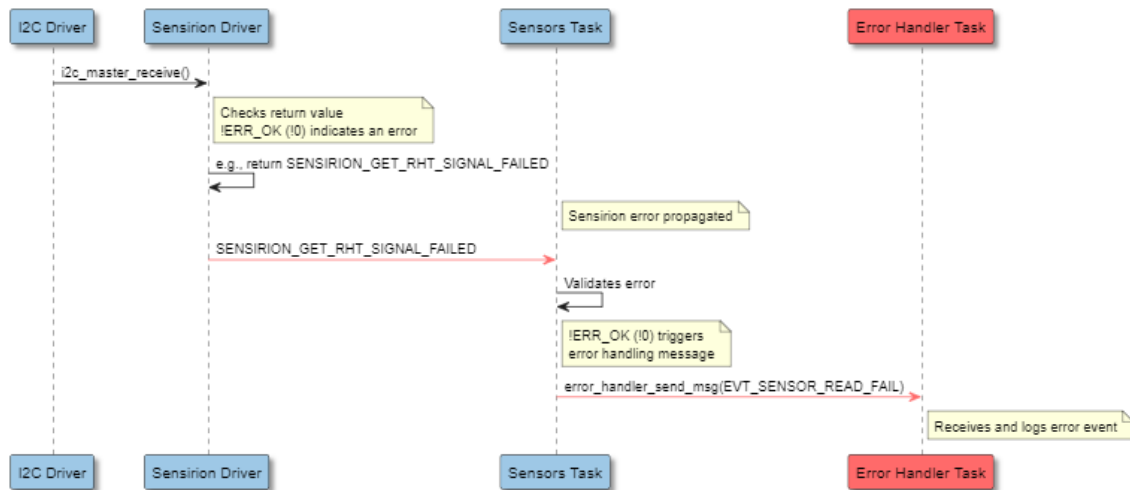
- **Event IDs:** Event IDs are passed as parameters to send message functions, enabling the Error Handler Task to parse and handle errors appropriately based on their type.
- **Project Significance:**
 - Ensures that all parts of the system can report back errors in a standardized way, which simplifies maintenance and scalability.



Error Handling & Error Handler Task Overview Part IV

Error Type Definition & Enumeration: Standardized error handling across the system.

- **Error Type:** `error_t` is a return type used throughout the system to indicate the status of operations, making error handling standardized across modules.
- **Error Enumeration:** `error_e` provides a comprehensive list of error codes.
- **Project Significance:** These error codes are used to propagate error information from lower-level hardware interactions up to higher-level management tasks.



Error Type Definitions Header File

Overview

- **Objective:** Define the `error_t` type and begin to establish a comprehensive set of error codes (`error_e`) for standardized error handling.
- **Key Steps:**
 - Create `error.h` to centralize error definitions.
 - Enumerate initial error codes that address common scenarios expected in the early stages of the project.
- **Project Significance:**
 - Provides immediate utility for early project phases, particularly in setting up basic drivers.

```
/**
 * Return type to be used for functions that require an error status returned.
 */
typedef int error_t;

/**
 * Enumeration of error constants (to be expanded as required).
 */
typedef enum
{
    ERR_OK = 0,
    ERR_FAIL = -1,
    ERR_TIMEOUT = -2,
    ERR_INVALID_PARAM = -3,
    SENSIRION_SHT_PROBE_FAILED = -4,
    SENSIRION_GET_RHT_SIGNAL_FAILED = -5,
    SENSIRION_SGP_PROBE_FAILED = -6,
    SENSIRION_GET_SGP_SIGNAL_FAILED = -7,
    SENSIRION_SET_RHT_SIGNAL_FAILED = -8,
    MODBUS_INVALID_REG_COUNT = -9,
    MODBUS_INVALID_REG_ADDRESS = -10,
    MODBUS_INVALID_END_ADDRESS = -11,
    MODBUS_INVALID_COIL_COUNT = -12,
    MODBUS_INVALID_COIL_ADDRESS = -13,
    MODBUS_INVALID_COIL_VALUE = -14,
    MODBUS_INVALID_DISC_COUNT = -15,
    MODBUS_INVALID_DISC_ADDRESS = -16,
    MODBUS_INVALID_DISC_VALUE = -17,
    MODBUS_MUTEX_NOT_CREATED = -18,
    MODBUS_MUTEX_TIMEOUT = -19,
    MODBUS_MUTEX_UNLOCK_FAIL = -20
} error_e;
```

Error Handler Header File

Overview

- **Objective:** Create definitions, enumeration and prototypes for the Error Handler Task, establishing the framework for systematic error management.
- **Key Steps:**
 - Define `EVT_BLINK_NUMBER` to for visual error indication during development.
 - Enumerate all possible error event identifiers (`event_id_e`).
 - Create `error_handler_send_msg` and `error_handler_send_msg_from_isr` function prototypes to handle error messaging for regular and interrupt contexts.
 - Provide a function prototype for starting the Error Handler Task, setting up essential configurations and task properties.
- **Project Significance:**
 - Provides an easily expandable interface for the Error Handler.

Error Handler Source File

Overview

- **Objective:** Implement the functionalities declared in the Error Handler Header File.
- **Key Steps:**
 - **Initialize Variables:** Set up queue, error count and active LED blinking variables.
 - **Task Initialization:** Write the startup sequence for the error handler task.
 - **Process Error Messages:** Write the task loop to receive and handle error messages based on predefined event IDs.
 - **Define LED Blink Behavior:** Implement a function to visually indicate errors.
 - **Error Message Functions:** Implement functions to send error messages to the handler, both normally and from ISR contexts.
- **Project Significance:** Enables quick diagnostics and debugging during development and allows for easy expansion or modification.

Error Handler Testing

Overview

- **Objective:** Demonstrate and verify the correct operation of the Error Handler Task.
- **Key Steps:**
 - Create a simple test in main.c; send various messages to the Error Handler Queue.
 - Start a debug session and verify that messages are received by the Error Handler Task and that the LED blinks as intended.
- **Project Significance:**
 - This practical test serves as an immediate check of our work, providing tangible evidence that the Error Handler Task can be used for debugging at early project development stages.

The background features a complex network of thin, light-colored lines and dots, forming various triangular shapes and a larger, interconnected web-like structure. The lines are primarily in shades of light green and yellow, while the dots are small and light grey. The overall effect is a subtle, technical, or mathematical aesthetic.

Next: Error Type Definitions Header File
